

3D GRAPHICS

Vuong Ba Thinh

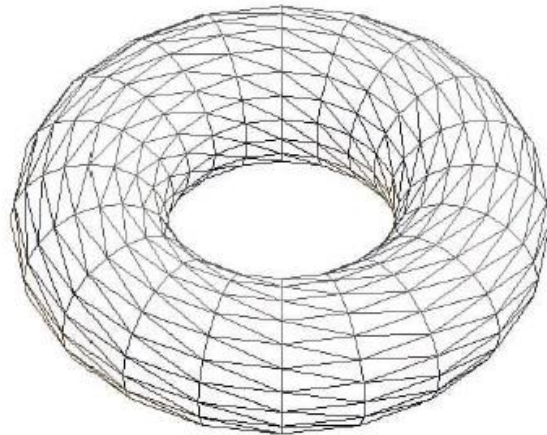
Department of CS –CSE Faculty - HCMUT

Outline

- Polygons
- Coordinate Spaces
- Lighting and Shading
- Visibility
- World Transform

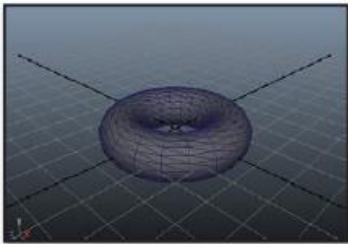
Polygons

- Triangles
- Graphical artifact
- Mesh – Model 3D

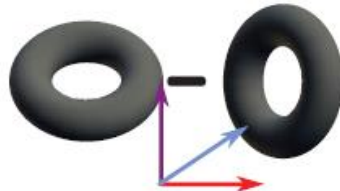


Coordinate Spaces

- Model
- World
- View/Camera
- Projection



(a)



(b)

Homogenous Coordinate

- Homogenous Coordinate: 4D coordinate is used for 3D space
- w-component:
 - 1: vector
 - 0: point

Transform 4D vector

- Multiply with 4x4 matrix

$$\begin{bmatrix} x' & y' & z' & w' \end{bmatrix} = \begin{bmatrix} x & y & z & w \end{bmatrix} \times \begin{bmatrix} a & e & i & m \\ b & f & j & n \\ c & g & k & o \\ d & h & l & p \end{bmatrix}$$

$$x' = x \cdot a + y \cdot b + z \cdot c + w \cdot d$$

$$y' = x \cdot e + y \cdot f + z \cdot g + w \cdot h$$

...

Transform Matrices

- Scale
- Translation
- Rotation

Scale matrix

$$s(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Translation Matrix

$$T(t_x, t_y, t_z) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix}$$

Rotation Matrix

$$\text{RotateX}(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{RotateY}(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{RotateZ}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Multiple Transform

$$\text{WorldTransform} = \text{Scale} \times \text{Rotation} \times \text{Translation}$$

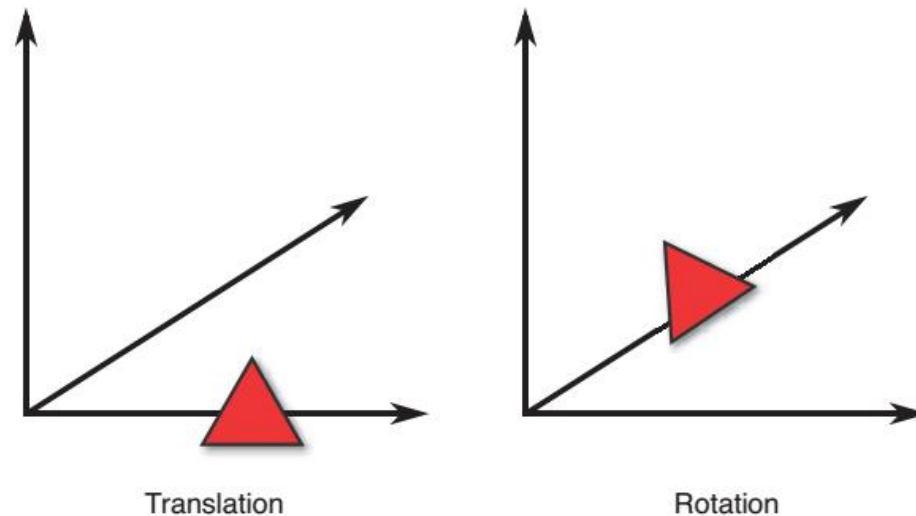
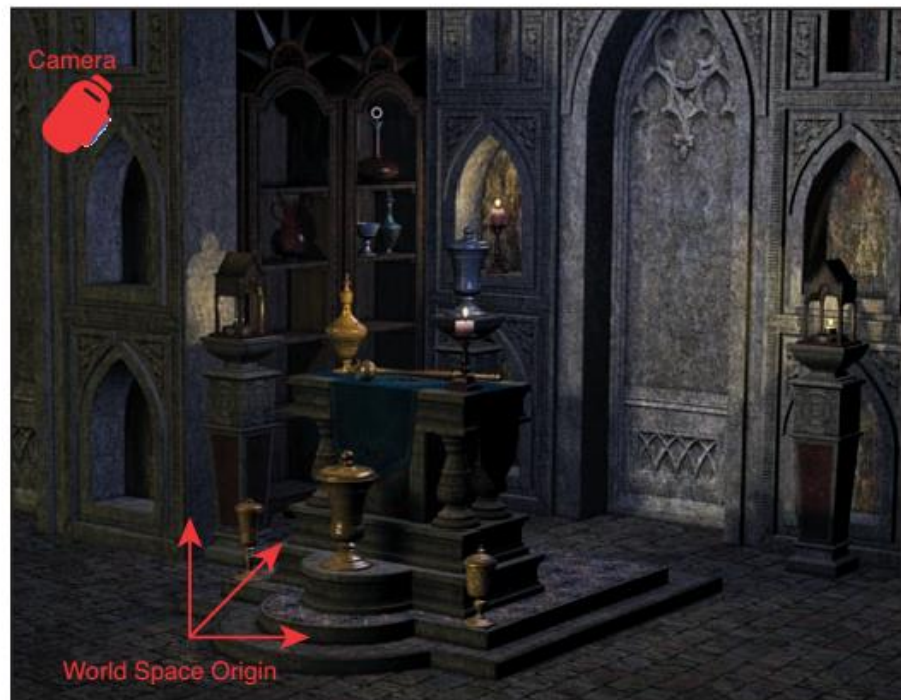


Figure 4.3 An object translated and then rotated about the world's origin instead of about its own origin.

View/Camera Space

$$\text{Look-At} = \begin{bmatrix} L_x & U_x & F_x & 0 \\ L_y & U_y & F_y & 0 \\ L_z & U_z & F_z & 0 \\ T_x & T_y & T_z & 1 \end{bmatrix}$$



View/Camera Space

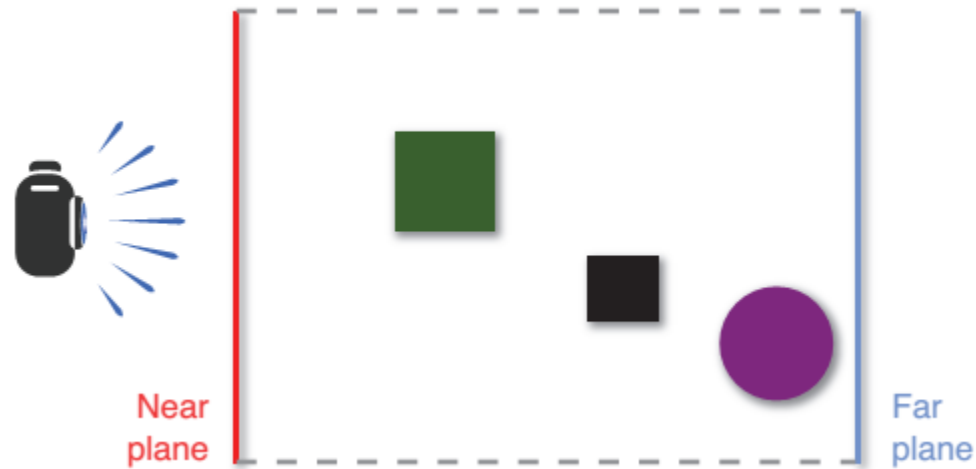
```
function CreateLookAt(Vector3 eye, Vector3 target, Vector3 Up)
    Vector3 F = Normalize(target - eye)
    Vector3 L = Normalize(CrossProduct(Up, F))
    Vector3 U = CrossProduct(F, L)
    Vector3 T
    T.x = -DotProduct(L, eye)
    T.y = -DotProduct(U, eye)
    T.z = -DotProduct(F, eye)

    // Create and return look-at matrix from F, L, U, and T
end
```

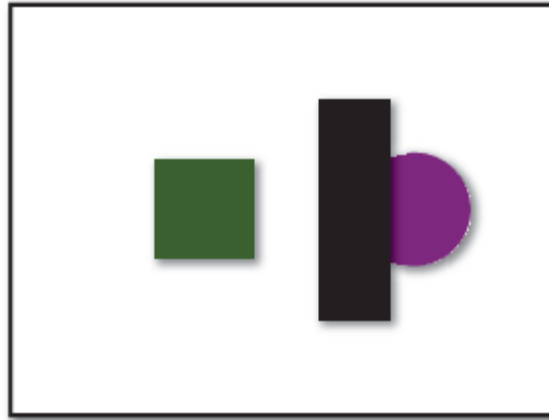
Projection Space

- 3D scene gets flattened into 2D scene.
- Orthographic projection
- Perspective projection

Orthographic Projection



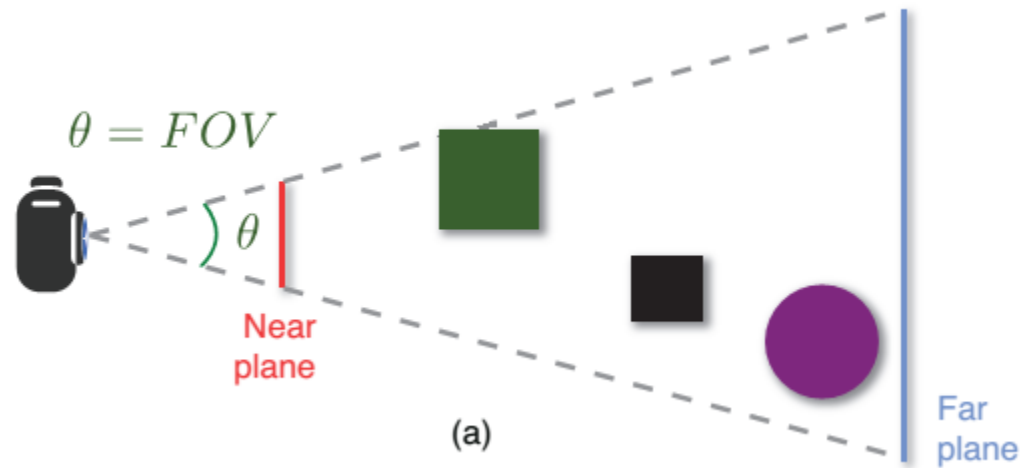
Orthographic Projection



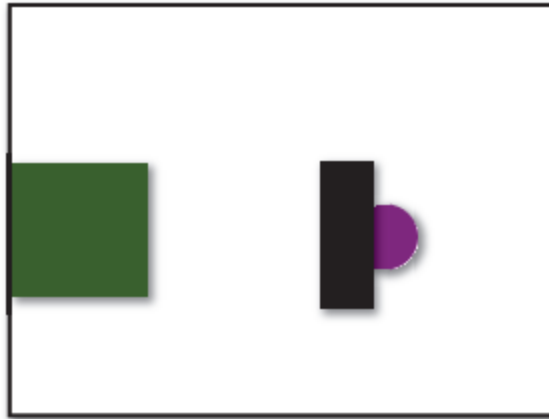
Orthographic Projection

$$\text{Orthographic} = \begin{bmatrix} \frac{2}{\text{width}} & 0 & 0 & 0 \\ 0 & \frac{2}{\text{height}} & 0 & 0 \\ 0 & 0 & \frac{1}{\text{far} - \text{near}} & 0 \\ 0 & 0 & \frac{\text{near}}{\text{near} - \text{far}} & 1 \end{bmatrix}$$

Perspective Projection



Perspective Projection



Perspective Projection

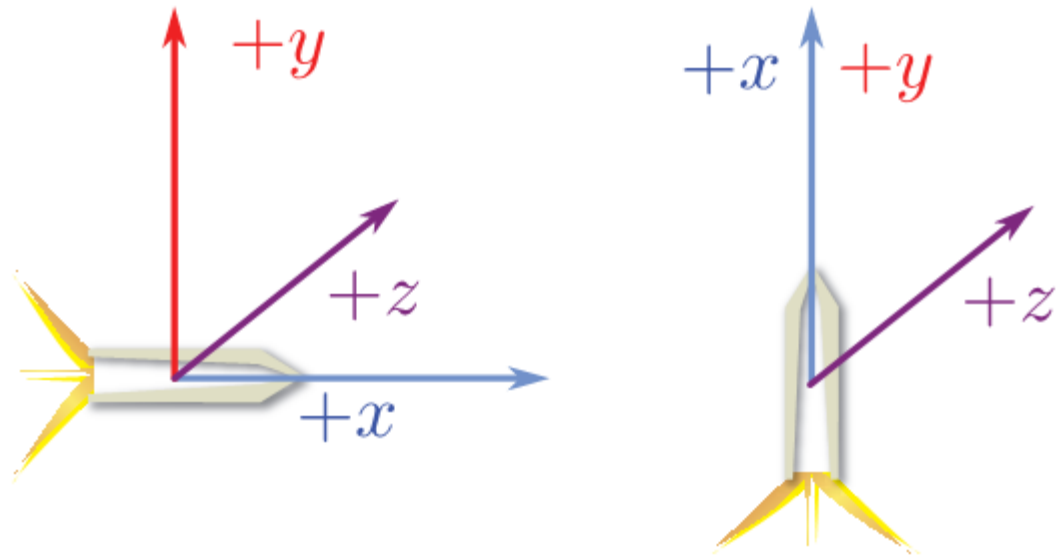
$$yScale = \cot(fov / 2)$$

$$xScale = yScale \cdot \frac{height}{width}$$

$$Perspective = \begin{bmatrix} xScale & 0 & 0 & 0 \\ 0 & yScale & 0 & 0 \\ 0 & 0 & \frac{far}{far - near} & 1 \\ 0 & 0 & \frac{-near \cdot far}{far - near} & 0 \end{bmatrix}$$

World Transform issues

- Memory
- Rotation



A 90° rotation about the z-axis causes gimbal lock.

Quaternion

- Rotate about an axis
- 2 quaternions can easily be interpolated between.

- Lerp

- Slerp

$$q_v = \hat{a} \sin \frac{\theta}{2}$$

- $q = [q_v, q_s]$

$$q_s = \cos \frac{\theta}{2}$$

- Rotate by q then $p \Rightarrow$ calculate pq .
- Grassmann product:

$$(pq)_v = p_s q_v + q_s p_v + p_v \times q_v$$

$$(pq)_s = p_s q_s - p_v \cdot q_v$$

- Quaternion $\rightarrow$ Matrix???

Lighting

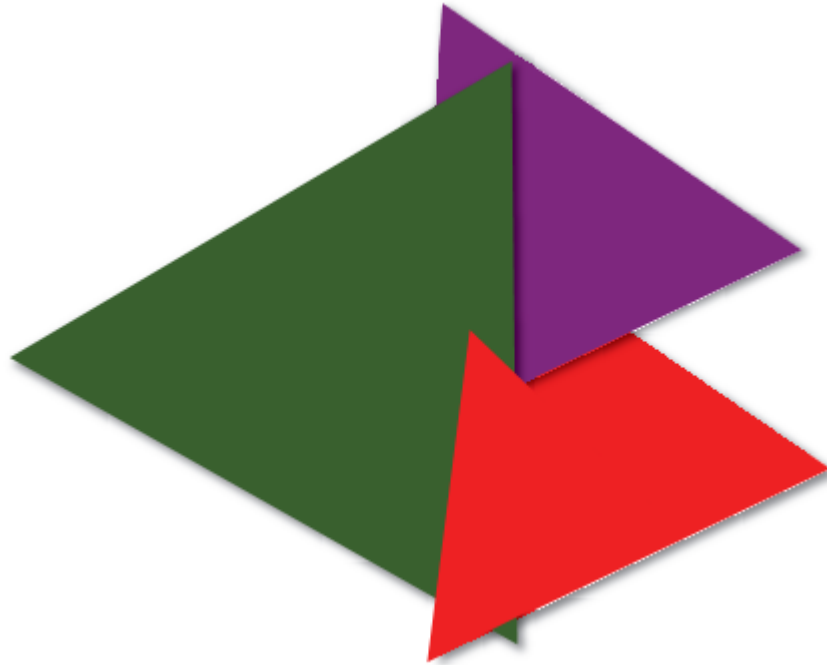
- Reading

Shading

- Reading

Visibility

- Painter's Alg
- Z-buffering



Scene



Z-Buffer

3D Game Object Representation

```
class 3DGameObject
    Quaternion rotation
    Vector3 position
    float scale

    function GetWorldTransform()
        // Order matters! Scale, rotate, translate.
        Matrix temp = CreateScale(scale) *
                        CreateFromQuaternion(rotation) *
                        CreateTranslation(position)

        return temp
    end
end
```

More Ref

- Page 92 in [1]